Malware / Trojan Attack Analysis & Response Platform

Module-specific workflow, tech stack and task backlog for a four-person team

Item	Detail
Team size	4 people (P1, P2, P3, P4), each owning two adjacent modules
Milestones	Six integration gates (G1-G6) that the modules must pass in order
Modules	M1 Ingestion, M2 Static, M3 Dynamic, M4 IOC, M5 Detection, M6 Response, M7 Core API, M8 UI/DevOps/QA
Method	Contract-first: one shared JSON schema, one event bus, stub modules from the start
Safety	Analysis runs only inside an isolated lab network; public corpora and benign test files only

1. System overview and ownership

The platform accepts a suspicious file, stores it safely, analyses it statically and dynamically, extracts indicators, scores the threat with explainable reasons, and runs (or proposes) response actions. Every module only reads and writes its own section of one shared result document, so the four owners can build in parallel and integrate through events.

Module	Project part	Owner	Depends on	Produces
M1 Ingestion & Vault	Intake	P1	M7 API	sample record, event sample.received
M2 Static Analysis	Analysis	P1	M1, rules from M5	static section, event static.done
M3 Dynamic Analysis	Analysis	P2	M1, lab network	dynamic section, pcap, event dynamic.done
M4 IOC & Network	Analysis	P2	M2, M3	iocs section, ATT&CK tags, event iocs.done
M5 Detection & Intel	Intelligence	P3	M2, M3, M4	detections, score, verdict, event scored
M6 Response Engine	Response	P3	M5, M7	response_actions, STIX/CSV exports
M7 Core API & DB	Platform	P4	none (others depend on it)	API, orchestrator, database
M8 UI, Reports, DevOps, QA	Presentation + infra	P4	M7 and all modules	dashboard, PDF report, CI/CD, tests

Pipeline and events

Stages run as workers listening on RabbitMQ queues. The orchestrator (M7) joins results and decides the next stage. Dynamic analysis is optional: if it fails or is skipped, scoring continues on static data and the report states that behaviour data is missing.

Event / queue	Published by	Consumed by	Payload
sample.received / q.ingest	M1	M7 (creates analysis)	sample_id, sha256, file_type, size
analysis.created / q.static	M7	M2	analysis_id, sample_id
static.done / q.dynamic	M2	M3 (if executable or macro document)	analysis_id, static summary flags
dynamic.done or dynamic.failed / q.ioc	M3	M4	analysis_id, artifact pointers (pcap, dropped files)
iocs.done / q.score	M4	M5	analysis_id
scored / q.response	M5	M6	analysis_id, verdict, score, tags
response.done / q.report	M6	M8 report worker	analysis_id, action ids
any failure / q.dlq	any	M7 (retry or mark FAILED)	stage, error, attempt

2. Shared contract (build together first)

Repository folder **/schemas** holds **analysis_result.schema.json** (JSON Schema draft 2020-12), versioned with semantic versioning. M7 validates every section write and rejects unknown properties. A change to the schema needs a pull request labelled **contract** approved by all four people.

Section ownership

Section	Writer	Required fields
sample	M1	sample_id, sha256, sha1, md5, ssdeep, size, file_type, original_name, submitted_at
static	M2	analyzer_version, format, entropy, strings_ref, imports_flags, yara_hits[], packer, suspicious_traits[]
dynamic	M3	status (ok, failed, skipped), process_tree[], files[], registry[], mutexes[], dropped[], pcap_ref
iocs	M4	items[]: type, value, defanged, source, confidence; attack[]: technique_id, evidence_ref; beaconing[]
detections / score / verdict	M5	contributions[]: reason, points, evidence_ref; score 0-100; verdict; family_tags[]; intel_hits[]
response_actions	M6	items[]: action, risk_level, status, approver, executed_at, dry_run, result

Example (abridged)

```
{
  "schema_version": "0.1.0",
  "analysis_id": "an_01H...",
  "sample": { "sample_id": "sm_01H...", "sha256": "e3b0c4...", "file_type": "PE32 executable", "size": 482304 },
  "static": { "format": "pe", "entropy": 7.42, "packer": "UPX (suspected)",
              "imports_flags": ["process_injection_set", "download_exec_set"],
              "yara_hits": [{ "rule": "loader_generic_01", "tags": ["loader"] }] },
  "dynamic": { "status": "ok", "process_tree": [{ "pid": 1812, "name": "sample.exe", "children": [] }],
               "registry": [{ "op": "set", "key": "HKCU\\...\\Run\\Updater" }] },
  "iocs": { "items": [{ "type": "domain", "value": "example-c2[.]test", "source": "pcap", "confidence": 80 }],
            "attack": [{ "technique_id": "T1547.001", "evidence_ref": "dynamic.registry[0]" }] },
  "score": 78, "verdict": "malicious",
  "contributions": [{ "reason": "Run key persistence", "points": 20, "evidence_ref": "dynamic.registry[0]" }],
  "response_actions": [{ "action": "export_blocklist", "risk_level": "medium", "status": "pending_approval" }]
}
```

Naming and conventions

- IDs are ULIDs with a prefix (sm_, an_, ac_). Timestamps are UTC ISO-8601. Hashes are lowercase hex.
- Large artifacts (pcap, dropped files, screenshots) are never embedded; sections hold a vault pointer such as **vault://sha256/...**.
- Every IOC is stored defanged for display (hxxp, [.]) and raw only in the database field used by exports.
- Every service exposes **GET /health** and logs JSON lines with a correlation id equal to analysis_id.

3. Module workflows

Tasks are listed in build order. The integration gates in section 4 decide whether a module is ready to be connected to the rest of the pipeline.

M1 Ingestion & Sample Vault

Owner: **P1**

Purpose. Receive files and archives safely, fingerprint them, deduplicate, store them encrypted, and start the pipeline.

Tech stack	Inputs / outputs
• Python 3.12, FastAPI, Uvicorn • python-magic, hashlib, ssdeep / tlsh • MinIO (S3 API) as vault, AES-256-GCM via cryptography • ClamAV (clamd) as one extra signal • RabbitMQ client (pika / aio-pika) • pytest	**In:** multipart upload from API (M7) or UI **Out:** sample record, encrypted object in vault, event sample.received

Task breakdown

ID	Task (specific)
M1-01	Scaffold service, Dockerfile (non-root, read-only filesystem), /health, config via environment
M1-02	POST /samples: stream upload, 64 MB cap, reject empty, sanitise name to [A-Za-z0-9._-], keep original only as metadata
M1-03	Streaming hashes: sha256, sha1, md5, ssdeep, tlsh without loading the file into memory
M1-04	True type with python-magic; flag extension mismatch (for example .pdf that is PE)
M1-05	Vault: MinIO bucket, object key = sha256, encrypt before write, key from secret store; no execute bit, no web access to bucket
M1-06	Dedupe: existing sha256 returns existing sample_id and records a new submission row
M1-07	Archive intake: zip with password 'infected'; limits depth 3, 1000 files, 500 MB expanded; extract in a temp dir inside the container only
M1-08	clamd scan result stored as a signal only, never as the verdict
M1-09	Publish sample.received; retry with backoff if broker is down
M1-10	Tests: EICAR, empty file, oversize, duplicate, zip bomb, path-traversal names, unicode names

Acceptance tests (module is done when all pass)

- Uploading the same file twice yields one sample_id and two submissions.
- A zip bomb and a path-traversal filename are rejected without side effects.
- Vault object cannot be read without the key, and the file on disk is never executable.
- Upload of a 50 MB file stays under 200 MB service memory.

Main risks and mitigations

- Key management mistakes: keep the key out of the repo and rotate through environment secrets.
- Real malware on a developer laptop: only use the vault from lab hosts.

M2 Static Analysis

Owner: **P1**

Purpose. Inspect a file without running it and produce structured findings the scorer can use.

Tech stack	Inputs / outputs
• Python, pefile, lief, capstone (optional disassembly) • yara-python with rules mounted read-only at /rules • FLOSS (FLARE) and strings for deobfuscated text • oletools (olevba, oleid) for Office • pdfid / peepdf for PDF • LnkParse3 for shortcuts • Detect-It-Easy CLI (packers) • Celery worker, pytest with golden files	**In:** event analysis.created, file from vault **Out:** static section, event static.done

Task breakdown

ID	Task (specific)
M2-01	File-type router: PE, Office/OOXML, PDF, script (PS1, JS, VBS, BAT), LNK, archive, other
M2-02	Common features: ASCII and UTF-16 strings (min length 6), FLOSS strings, whole-file and per-section entropy
M2-03	PE analyzer: compile timestamp, sections (name, sizes, entropy, flags), imports, exports, imphash, resources, overlay, signature present
M2-04	Suspicious import sets as named flags: process_injection_set, download_exec_set, keylog_hook_set, persistence_set, anti_debug_set
M2-05	Packer check: DIE wrapper plus heuristic (entropy above 7.0 and fewer than 10 imports)
M2-06	YARA scan against /rules with a 30 s timeout per file; return rule, tags, meta
M2-07	Office: olevba auto-exec macros, suspicious keywords, external relationships in OOXML
M2-08	PDF: /JS, /OpenAction, /Launch, embedded files counts
M2-09	Scripts: obfuscation indicators (long base64, char-code joins, encoded command flags), URL extraction; never executed
M2-10	LNK and archives: target path and arguments, double extensions, nested archive listing
M2-11	Emit static section, validate against schema, publish static.done
M2-12	Corpus tests: 30 benign (system tools, installers, office files) and 30 public samples in the lab; golden JSON snapshots; under 30 s per 50 MB

Acceptance tests (module is done when all pass)

- Output validates against the schema for every corpus file, including corrupt and truncated files.
- Analyzer never raises an unhandled exception: failures become static.errors[] and the pipeline continues.
- Benign corpus produces no more than 2 percent YARA malware-family hits.
- Re-running the same file gives identical output (deterministic).

Main risks and mitigations

- Parser crashes on malformed files: fuzz with truncated inputs and wrap in a subprocess with memory limit.
- Rule false positives: coordinate rule tuning with P3 in the tuning session.

M3 Dynamic (Sandbox) Analysis

Owner: **P2**

Purpose. Run samples inside an isolated Windows guest, observe behaviour, and normalise the results.

Tech stack	Inputs / outputs
• CAPEv2 (or Cuckoo3) on Ubuntu host • KVM/libvirt with an isolated network (no NAT, no route to LAN) • Windows 10 guest, Sysmon, CAPE agent • INetSim (or FakeNet-NG) on a separate Linux VM • tcpdump / CAPE pcap • Python worker (requests to CAPE REST API), pytest	**In:** event static.done, sample from vault **Out:** dynamic section, pcap and dropped files in vault, event dynamic.done

Task breakdown

ID	Task (specific)
M3-01	Lab network: isolated libvirt network, host firewall drops forwarding, guest has no shared folders or clipboard, snapshot 'clean'
M3-02	Install CAPEv2, agent, Sysmon config; INetSim answering DNS, HTTP, HTTPS, SMTP
M3-03	Submission worker: consume q.dynamic, choose timeout (default 120 s, max 300 s), submit via REST
M3-04	Polling and failure handling: VM boot failure, agent timeout; 2 retries then dynamic.status=failed with reason
M3-05	Normalise CAPE report.json: process tree, file create/modify/delete, registry set, mutexes, dropped file hashes, signature hits
M3-06	Move pcap and dropped files into the vault (encrypted), store pointers only
M3-07	Health checks: canary file hash after revert, guest clock, disk space; scheduled canary run
M3-08	Benign behaviour simulator (small program that writes a file, sets a Run key, makes a DNS and HTTP call to INetSim) used as a test fixture
M3-09	Publish dynamic.done / dynamic.failed; skip logic for non-executable types
M3-10	Runbook: how to rebuild the lab from scratch

Acceptance tests (module is done when all pass)

- EICAR and the benign simulator both complete; the simulator's file, registry and DNS actions appear in the normalised output.
- After every run the guest reverts and the canary hash matches.
- No packet from the guest ever reaches the real LAN (verified with a capture on the host uplink).
- A failed run does not block the pipeline; the report shows 'behaviour data unavailable'.

Main risks and mitigations

- Sandbox escape or network leak: isolated network, no bridged adapter, patch hypervisor, test egress regularly.
- Evasive samples that detect VMs: record it as a finding, do not try to defeat it.

M4 IOC & Network Extraction

Owner: **P2**

Purpose. Turn raw strings, behaviours and packet captures into deduplicated indicators and ATT&CK tags.

Tech stack	Inputs / outputs
• Python, iocextract, validators, tldextract • Zeek (container) for pcap to logs, pyshark as fallback • mitreattack-python for technique names • pandas for interval statistics • pytest with saved pcap fixtures	**In:** static and dynamic sections, pcap from vault **Out:** iocs section, event iocs.done

Task breakdown

ID	Task (specific)
M4-01	IOC model: type (ipv4, ipv6, domain, url, email, sha256, mutex, regkey, filepath, user_agent), value, defanged, source, confidence 0-100
M4-02	Extractors from static strings and dynamic artifacts; drop private IPs and an allowlist of known benign domains
M4-03	Run Zeek on pcap; parse dns.log, http.log, ssl.log, conn.log into DNS queries, HTTP requests (host, URI, UA), SNI, destination ip:port
M4-04	Beacon heuristic: same destination, at least 5 connections, interval jitter below 20 percent, tag possible_beaconing
M4-05	Merge and dedupe; confidence rules (seen in static and dynamic raises it, string-only lowers it)
M4-06	Behaviour to ATT&CK table: Run key or startup folder T1547.001, scheduled task T1053.005, service creation T1543.003, process injection T1055, tool transfer T1105, keylogging T1056.001, LSASS access T1003.001, application layer protocol T1071
M4-07	Emit iocs section and iocs.done; export helper for M6 (list by type)
M4-08	Tests using fixtures generated by the M3 benign simulator and public sample pcaps

Acceptance tests (module is done when all pass)

- Every IOC has type, source and confidence; no duplicates after merge.
- Benign simulator produces its known domain, URL, Run key and file path as IOCs and the correct T-IDs.
- Beacon heuristic flags a scripted 30-second interval fixture and not a normal browsing pcap.

Main risks and mitigations

- Noisy IOCs (CDNs, update servers): keep the allowlist in a versioned file reviewed by P3.
- Large pcaps: cap at 100 MB and sample if exceeded.

M5 Detection, Scoring & Threat Intel

Owner: **P3**

Purpose. Combine static, behavioural and intelligence signals into an explainable verdict.

Tech stack	Inputs / outputs
• YARA and Sigma rules in /rules (git), yara CLI and sigma-cli in CI • Python, scikit-learn, LightGBM, pandas • EMBER dataset (PE static features) for the classifier • VirusTotal, MalwareBazaar, AlienVault OTX APIs; MISP optional • Redis cache, tenacity for retries • pytest, evaluation scripts	**In:** static, dynamic, iocs sections **Out:** detections, score, verdict, family_tags; event scored

Task breakdown

ID	Task (specific)
M5-01	Rule repository layout: rules/yara/community, rules/yara/custom, rules/sigma; CI compiles every rule; record each source's license
M5-02	Custom YARA for Trojan classes by capability (loader, downloader, RAT command handling, credential stealer paths), each with meta and tags
M5-03	Sigma rules over Sysmon-style events (Run key set, suspicious parent-child, script host spawning shells)
M5-04	Intel client: hash, domain, IP lookups; Redis cache 24 h; respect rate limits (VT free tier 4 per minute); circuit breaker; offline mode
M5-05	Scoring v1: points table, cap 100, verdict thresholds 0-29 benign, 30-59 suspicious, 60-100 malicious
M5-06	Explainability: every point recorded as reason, points, evidence_ref
M5-07	Trojan subtype tags (dropper, loader, RAT, stealer, banker) from rule tags with confidence
M5-08	Join logic: score when iocs.done arrived and static present; dynamic optional
M5-09	ML v2: EMBER features, LightGBM, train/validation split, calibrated probability, compare with v1 on held-out data
M5-10	Evaluation harness: labelled CSV, confusion matrix, precision, recall, false-positive rate; report after every rule or model change
M5-11	Final ensemble choice and written evaluation (limits, dataset bias, drift)

Acceptance tests (module is done when all pass)

- Scoring v1 point table (starting values): family YARA hit +40, injection import set +10, packed or high entropy +10, Run key persistence +20, injection behaviour +20, beaconing +20, intel hit with 5 or more engines +30.
- Targets on the team corpus: false-positive rate below 2 percent on benign files, recall above 85 percent on known samples.
- Every verdict has at least one contribution with a valid evidence_ref.
- Intel outage does not fail the stage: result says 'intel unavailable'.

Main risks and mitigations

- API quotas: cache aggressively and batch lookups.
- Overfitting to a small corpus: keep a held-out set nobody tunes against.

M6 Response Engine

Owner: **P3**

Purpose. Choose and run the right response actions from a verdict, with approval and a tamper-evident audit trail.

Tech stack	Inputs / outputs
• Python, Celery workers • YAML playbooks validated with pydantic • Slack/Teams webhooks, SMTP, GitHub or Jira issues API • Plain-text blocklist feed served by M7 • stix2 library for STIX 2.1 • FastAPI mock server as EDR/firewall stub • pytest	**In:** scored event, verdict, tags, IOCs **Out:** response_actions section, exports, event response.done

Task breakdown

ID	Task (specific)
M6-01	Action catalog with risk: LOW (tag, watchlist, notify, ticket), MEDIUM (publish blocklist, block hash in EDR stub), HIGH (isolate host, kill process, delete file)
M6-02	Playbook schema: trigger (verdict, min_score, tags), steps (action, params, requires_approval); write playbooks generic-malicious, trojan-rat, trojan-stealer, suspicious-review
M6-03	Playbook engine: match, create action rows (pending, approved, executed, failed, skipped), idempotent execution
M6-04	Connectors with dry-run flag: webhook, email, issue tracker, blocklist feed, EDR stub
M6-05	Approval flow: HIGH always needs approver role; pending actions expire after 24 h
M6-06	Audit log: append-only table, each row stores hash of previous row; verification script
M6-07	Exports: STIX 2.1 bundle (indicator, malware, attack-pattern, relationship), CSV, JSON
M6-08	Remediation text per tag (isolate, preserve evidence, hunt for IOCs, reimage, rotate credentials) included in the report
M6-09	Tests: dry-run of each playbook, connector failure with backoff, re-delivery does not duplicate actions

Acceptance tests (module is done when all pass)

- A malicious verdict with RAT tag triggers the trojan-rat playbook in dry-run and lists exactly its steps.
- A HIGH-risk step cannot execute without an approver; the audit chain verifies after the run.
- The STIX bundle validates with the stix2 validator and imports into a MISP test instance (optional).

Main risks and mitigations

- Dangerous automation: default to dry-run, real mode only for the EDR stub in this project.
- Alert fatigue: one notification per analysis, not per action.

M7 Core API, Orchestration & Database

Owner: **P4**

Purpose. Own the schema, API, workflow state machine, security and persistence for everything else.

Tech stack	Inputs / outputs
• FastAPI, Pydantic v2, SQLAlchemy 2, Alembic • PostgreSQL 16 (JSONB for sections) • RabbitMQ + Celery, Redis • JWT (python-jose) with argon2 password hashing • jsonschema validation • Prism mock server • pytest, schemathesis	**In:** requests from UI and events from modules **Out:** persisted analyses, API, orchestration decisions

Task breakdown

ID	Task (specific)
M7-01	Write OpenAPI spec and JSON Schema; start Prism mock server so others can integrate immediately
M7-02	Database tables: users, samples, submissions, analyses, analysis_sections (jsonb), iocs, detections, actions, audit_log, reports; Alembic migrations
M7-03	State machine: RECEIVED, STATIC_RUNNING, STATIC_DONE, DYNAMIC_RUNNING, DYNAMIC_DONE or DYNAMIC_FAILED or DYNAMIC_SKIPPED, IOC_DONE, SCORED, RESPONDING, COMPLETE, FAILED
M7-04	Stub workers for every stage so an end-to-end run exists at gate G2
M7-05	Internal API: PUT /internal/analyses/{id}/sections/{name} with schema validation and service API keys
M7-06	Public API: POST /api/v1/samples, GET /api/v1/samples/{id}, GET /api/v1/analyses/{id}, GET /api/v1/iocs, POST /api/v1/actions/{id}/approve, GET /api/v1/reports/{id}.pdf
M7-07	Auth: roles analyst, approver, admin; short-lived JWT; refresh; lockout after repeated failures
M7-08	Reliability: idempotency key analysis_id+stage, retries with backoff, dead-letter queue, admin requeue endpoint
M7-09	Search and filters by hash, verdict, date range, IOC value, tag; pagination
M7-10	Rate limiting, request size limits, structured logs, metrics endpoint
M7-11	Backups (pg_dump script), retention policy setting for samples and reports
M7-12	Load test: 50 concurrent submissions, find and fix bottlenecks

Acceptance tests (module is done when all pass)

- A sample can pass the whole pipeline with stub stages before the real modules are integrated.
- Invalid section writes return 422 with the failing JSON path; unknown fields are rejected.
- Schemathesis finds no 5xx errors against the OpenAPI spec.
- Killing any worker mid-stage results in a retry, not a lost analysis.

Main risks and mitigations

- Schema churn: freeze v0.1 at G1 and use additive changes only after that.
- Single point of failure: P1 is the named backup for M7 reviews.

M8 Dashboard, Reporting, DevOps & QA

Owner: **P4**

Purpose. Give analysts a usable interface and reports, and keep the whole project buildable, tested and deployable.

Tech stack	Inputs / outputs
• React 18 + Vite + TypeScript, Tailwind CSS • TanStack Query, Recharts / D3 (timeline, ATT&CK heatmap) • Jinja2 + WeasyPrint for PDF reports • Docker Compose (profiles: core, analysis, lab) • GitHub Actions, ruff, black, eslint, Trivy • pytest, Playwright, schemathesis, Prometheus + Grafana	**In:** API from M7 **Out:** web UI, PDF reports, CI/CD, regression results

Task breakdown

ID	Task (specific)
M8-01	Monorepo, pre-commit hooks, CI pipeline: lint, unit tests, schema validation, image build, Trivy scan
M8-02	Docker Compose profiles; lab compose kept separate for the isolated host
M8-03	UI shell: login, role-aware navigation, upload page with progress and error states
M8-04	Analyses list with filters; sample detail with tabs Overview, Static, Behaviour, Network, IOCs, ATT&CK, Response, Audit
M8-05	Visuals: process tree, behaviour timeline, ATT&CK heatmap, score breakdown bar
M8-06	Response tab: pending actions, approve / reject with reason, status updates
M8-07	PDF report: executive summary, verdict and score reasons, key behaviours, IOC tables, ATT&CK, recommended actions, hash appendix
M8-08	Regression: Playwright end-to-end flow, scheduled corpus run, metric trend chart
M8-09	Observability: queue depth, per-stage throughput, failure rate dashboards
M8-10	Security review of the platform: upload handling, authz tests, secrets, container hardening checklist
M8-11	Docs: README, architecture diagram, lab runbook link, user guide, final demo script

Acceptance tests (module is done when all pass)

- A non-developer can upload a sample and explain the verdict and next steps from the UI and PDF alone.
- CI blocks merges on lint, test, schema, or critical vulnerability failures.
- Playwright flow upload to report passes on a clean checkout with one command.
- Every UI state has loading, empty and error handling.

Main risks and mitigations

- UI waits on backend: build against the Prism mock from the start.
- Report layout drift: snapshot-test the generated PDF text.

4. Integration gates

Gate	Must be demonstrable	Pass rule
G1	Schema v0.1 and OpenAPI published, Prism mock running, repo and CI green, lab network isolated, rule repo compiling	All four sign off on the schema
G2	Upload through UI, stub stages, status updates, stub verdict displayed; M1 real and storing in vault	10 uploads run without a lost analysis
G3	Real M2 (PE + YARA) replaces stub; M5 scoring v1; M3 running the benign simulator	Static verdict with reasons shown on the UI
G4	Real M3 and M4 in the pipeline; behaviour, IOC and ATT&CK tabs populated; PDF draft	Full report without response actions
G5	M5 v2 evaluated, M6 playbooks and approvals, STIX export, audit chain, report complete	End-to-end with response in dry-run
G6	Feature freeze, security review fixes, docs, final regression, demo	No open critical or high defects

Build order per person

Stage	P1 (M1, M2)	P2 (M3, M4)	P3 (M5, M6)	P4 (M7, M8)
Up to G1	Shared schema; M1-01 to M1-04	M3-01, M3-02 lab and CAPE	M5-01 rule repo; intel source list	M7-01, M7-02, M8-01, M8-02
G1 to G2	M1-05 to M1-10; M2-01, M2-02	M3-02 finish; M3-08 simulator	M5-02 first YARA set	M7-03 to M7-05 stubs; M8-03 UI shell
G2 to G3	M2-03 to M2-06	M3-03 to M3-05; M4-01	M5-04 to M5-06 intel and scoring v1	M7-06, M7-07; M8-04 begin
G3 to G4	M2-07 to M2-12	M3-05 to M3-10; M4-02 to M4-07	M5-03, M5-07, M5-08; M6-01 to M6-04	M7-08, M7-09; M8-04, M8-05, M8-08 start
G4 to G5	Rule tuning support; M1 hardening	M4-08; lab hardening	M5-09, M5-10; M6-05 to M6-07	M7-10; M8-06, M8-07, M8-09
G5 to G6	Docs, defect fixes, regression	Docs, defect fixes, regression	M5-11; M6-08, M6-09; evaluation write-up	M7-11, M7-12; M8-10, M8-11; freeze and demo

Cross-module reviewers

Author	Primary reviewer	Why
P1 (M1, M2)	P2	Both feed analysis data downstream; P2 consumes P1's output
P2 (M3, M4)	P1	Needs to understand artifact formats the static side also uses
P3 (M5, M6)	P4	P4 owns the API and UI that expose verdicts and approvals
P4 (M7, M8)	P3	P3 depends on the sections API and the approval flow

5. Working agreements

Team sync points

Sync	Purpose
Planning sync	Each person: goals, blockers, interface changes needed
Integration check	Run the shared end-to-end script, fix contract drift
Demo and tuning session	Demo finished module work, rule and score tuning (P3 with P1 and P2), retro

Git and pull request rules

- Trunk-based: branches named **m2/pe-parser**, **m5/scoring-v1**, short-lived, squash merge.
- PR template asks: module, task ID, contract impact (yes/no), tests added, screenshots (UI), safety impact.
- CI must be green, plus one approval from the cross-module reviewer (section 4). Contract PRs need all four approvals.
- No sample files, keys or credentials in git. Samples live only in the lab vault; fixtures in git are benign or EICAR.
- Definition of done per task: code, unit tests, schema validation, doc entry, reviewed, demoed in the demo session.

Integration habits

- Each module ships a **make e2e-local** target that feeds a known benign sample through its own stage with the others mocked.
- At every integration check the full-stack script runs EICAR, the benign simulator and one document sample; results are posted in the team channel.
- If a module falls behind, the owner flags it at the planning sync and the gate owner decides whether to cut scope or reassign tasks.

6. Lab and safety rules

- All execution of samples happens only in the lab guest. Never open samples on development laptops or CI runners.
- Lab network: isolated virtual network, no NAT, no bridge, no shared folders or clipboard; verify with a host-side packet capture regularly.
- Use public research corpora and benign or EICAR test files. The team does not write, modify or weaponise malware; the project only analyses and responds.
- Samples are stored encrypted and referenced by hash; downloads from the UI are archives with password 'infected'.
- IOCs are shown defanged. Destructive response actions are opt-in, require an approver, and are dry-run by default.
- Treat the platform as a target: size limits, type checks, non-root containers, read-only filesystems, secrets from environment, dependency scanning in CI.

7. Risk register

Risk	Likelihood	Impact	Owner	Mitigation
CAPE install or guest setup is harder than expected	High	High	P2	Start first; fallback is static-only demo with recorded dynamic reports
Schema changes break modules after integration	Medium	High	P4	Freeze at G1; additive changes only; contract tests in CI
False positives on benign software	Medium	Medium	P3	Benign corpus in CI; allowlists; regular tuning session
Third-party API limits or outages	High	Low	P3	Cache, circuit breaker, offline mode
Sandbox network leak	Low	Critical	P2	Isolated network, regular egress test, no bridged adapters
One person becomes a bottleneck	Medium	High	All	Named reviewer pairs; stubs let others continue

8. Final deliverables checklist

- Running stack from one command (core profile) plus documented lab setup.
- Web UI and PDF report for at least 10 analysed samples (benign, EICAR, public samples).
- Evaluation write-up with confusion matrix, false-positive rate and known limitations.
- Playbooks, STIX export sample, audit verification output.
- Architecture diagram, runbooks, user guide, and demo script.